



北京航空航天大学
BEIHANG UNIVERSITY

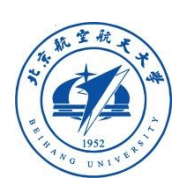


微机原理与接口技术 第四章

林 新

Lx@buaa.edu.cn

北京航空航天大学 自动化学院

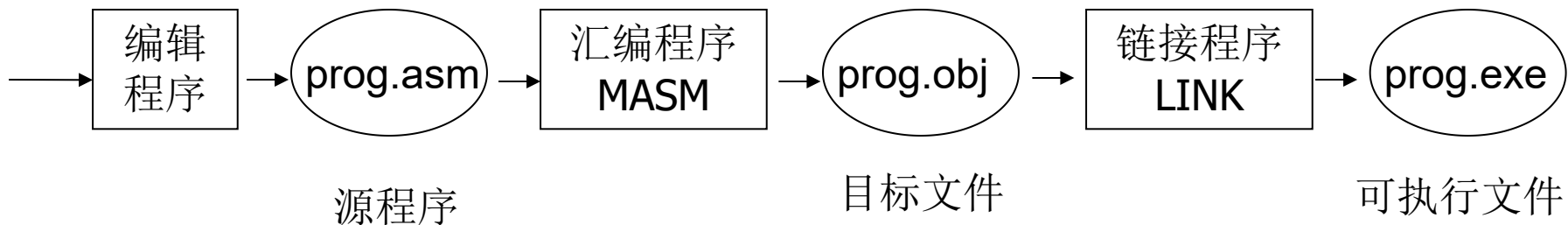


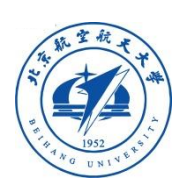
主要内容

1. 汇编语言程序的组成：指令和伪指令的基本书写格式。
2. 常用伪指令：数据定义，符号定义，指定存贮单元类型，段定义（定位类型及连接类型），段寄存器说明，过程(子程序)定义，多模块间相互调用，源程序结束。
3. 汇编语言程序的执行过程：汇编语言源程序的编写、汇编(MASM.EXE的使用)与连接(LINK.EXE的使用)、运行及调试工具软件DEBUG.COM常用命令的使用
4. 常用DOS子程序(INT 21H的1、2、9、A号功能)调用，入口参数，出口参数。
5. 不同结构程序设计举例：简单程序，分支程序，循环程序，子程序设计（主程序+子程序、主程序+子程序+宏），多模块程序设计。



汇编程序编写、编译和运行





4.1 汇编语言程序格式和伪指令

4.1.1 汇编语言程序格式

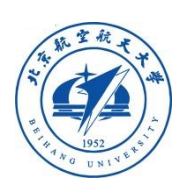
汇编语句一行只能写一条汇编语句，一条汇编语句也只能写在一行内。分为指令性语句和伪指令语句。

■指令语句

- 编译器转换为二进制代码后由CPU执行

■伪指令语句

- 告诉编译器如何进行编译，**没有**对应的机器码，因此也就不会由CPU执行



4.1.1 汇编语言程序格式

一、指令性语句

例

NEXT: TEST AL, 80H ;测试D7位

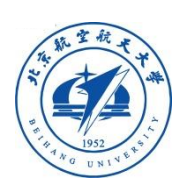
[标号 :] 指令助记符 [操作数 [, 操作数]] [; 注释]

1) 标号

■ 标号的取名规则:

- 可由字母、数字、下划线等特殊字符组成
- 首字符必须为字母或某些特殊字符，不能为数字（数值常量必须以数字开头）
- 名字最长31个字符
- 保留字不能作为名字

■ 标号具有3种属性：段基址，偏移地址，类型。



4.1.1 汇编语言程序格式

一、指令性语句

例

NEXT: TEST AL, 80H ;测试D7位

[标号 :] 指令助记符 [操作数 [, 操作数]] [; 注释]

2) 指令助记符

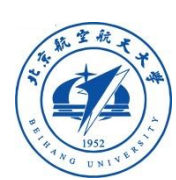
指令语句中必须有的部分，指明**CPU**的操作

3) 操作数项

可以是常数、寄存器、变量（存储单元）、标号、表达式，其中常数、变量和标号是三种基本数据项。

4) 注释

用于说明指令或程序的功能，增强程序可读性。



4.1.1 汇编语言程序格式

常数：具体的数据，可以是数字常量或字符常量。

- 数字默认十进制，也可加**D**表示十进制数。如 1234D, 1234
- 数字后加**B**表示二进制数。如 1010B
- 数字后加**H**表示十六进制数。如 1234H
- 字符常量，用单引号表示。如 '1234'、'abc'

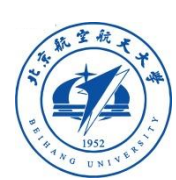
字符常量汇编后，用字符对应的ASCII码表示

- A、B、C、D、E、F开头的十六进制数前面加0，与H结尾的标识符区别。如 常量0AH, 0CHH, 寄存器名AH、BH、CH、DH, 变量名 abcdH 等

变量：用符号表示的存放在存储单元中的可变数值。由DB、DW、DD等伪指令定义。

- 具有三种属性：段基址、偏移地址、类型

```
例  data1  DB  12, 34, 56          ; 十进制
     data2  DB  12H, 34H, 56H       ; 十六进制
     STR1   DW  'A','B'             ; 字符
     string DB  '1234'              ; 字符串
```



4.1.1 汇编语言程序格式

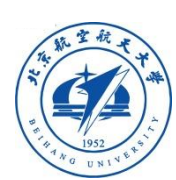
二、伪指令语句

例

STK1 SEGMENT STACK ;堆栈段

[名字] 伪指令指示符 [操作数] [; 注释]

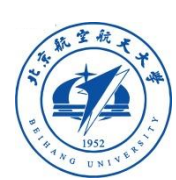
- ① 名字
- ② 伪指令指示符
- ③ 操作数
- ④ 注释



4.1.1 汇编语言程序格式

三、表达式和运算符

- 常数、符号通过运算符连接起来的式子称为**表达式**。表达式的计算不是由CPU完成，而是在汇编时由编译程序完成。
- 表达式所使用的运算符：
 - 算术运算符：+, -, *, /, MOD, SHL, SHR
 - 逻辑运算符：AND, OR, XOR, NOT
 - 关系运算符：EQ, NE, LT, LE, GT, GE
 - 数值返回：OFFSET, SEG, TYPE, SIZE...
 - 修改属性：PTR, THIS, HIGH, LOW, SHORT....
 - 其它：(), [], ,, < >, ...



4.1.1 汇编语言程序格式--表达式和运算符

(1) 算术运算符

- 包括 +, -, *, /, MOD, SHL, SHR
- 运算的结果和对象都是整数。

例

```
ARRAY      DB 1,2,3,4,5,6,7,8
TRY        DB 20
```

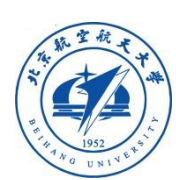
```
MOV AX, 30*5
```

```
MOV CX, (TRY-ARRAY) ;变量的偏移地址相减
                    ;即数组长度存入CX
```

;上述语句等效于:

```
MOV AX, 150
MOV CX, 8
```

		偏移地址
ARRAY	1	0000H
	2	0001H
	3	0002H
	4	0003H
	5	0004H
	6	0005H
	7	0006H
	8	0007H
TRY	20	0008H



4.1.1 汇编语言程序格式--表达式和运算符

补充：现行地址符\$（程序计数器）

- \$表示位置的偏移地址

- 例4.1

```
DATA          SEGMENT
DT1           DB      21H
LIST          DB      12,38,5,29,74
CONT          EQU     $-LIST
DT2           DB      'E'
DATA          ENDS
```

...

```
MOV          CX, COUNT; 数组长度->CX
```

		偏移地址
DT1	21H	0000H
LIST	0CH	0001H
	26H	0002H
	05H	0003H
	1DH	0004H
	4AH	0005H
		0006H \$=0006H



4.1.1 汇编语言程序格式--表达式和运算符

■ 例4.1

```
DATA          SEGMENT
DT1           DB      33
LIST          DB      12,38,5,29,74
CONT          EQU     $-LIST
DT2           DB      'E'
DATA          ENDS
```

❑ COUNT = 当前\$所在位置的地
址-LIST的偏移地址

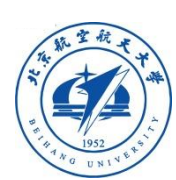
❑ EQU定义的常量**不占用**内存

偏移地址		
DT1	21H	0000H
LIST	0CH	0001H
	26H	0002H
	05H	0003H
	1DH	0004H
	4AH	0005H
CONT	05H	0006H \$=0006H
DT2	45H	0007H



偏移地址		
DT1	21H	0000H
LIST	0CH	0001H
	26H	0002H
	05H	0003H
	1DH	0004H
	4AH	0005H
DT2	45H	0006H \$=0006H
		0007H





4.1.1 汇编语言程序格式--表达式和运算符

(2) 逻辑运算符和关系运算符

■ AND, OR, XOR, NOT

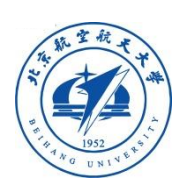
- 只能对常数进行运算，得到的结果也是常数
- 与指令的区别

例: `MOV AL, 1100 1100B AND 10001000B`
等效于 `MOV AL, 1000 1000B`

■ EQ, NE, LT, LE, GT, GE

- 关系运算符的两个操作数必须是数据，或是同一段内的两个存储单元的地址。
- 运算结果是一个数值，若为真，输出全是1，即0FFH或0FFFFH；若为假，则输出全是0

例: `MOV AL, 100 GT 32H`
等效于 `MOV AL, 0FFH`



4.1.1 汇编语言程序格式--表达式和运算符

(3) 数值返回

加在变量或标号前，返回运算对象的某个参数值。

- OFFSET：返回偏移地址
- SEG：返回段基址
- TYPE：返回变量类型属性
- LENGTH：返回DUP前面的数，如果是变量则返回1
- SIZE：等于TYPE*LENGTH

TYPE返回类型

	类型	返回值
变量	DB	1
	DW	2
	DD	4
	DQ	8
	DT	10
标号	NEAR	-1 (FFH)
	FAR	-2 (FEH)



4.1.1 汇编语言程序格式--表

(3) 数值返回

例：数据段段基址为2100H

```
DATA SEGMENT
```

```
    M1    DW    100 DUP(?)
```

```
    M2    DW    1,2,3
```

```
    M3    DB    'ABCD'
```

```
DATA ENDS
```

- OFFSET,SEG,
- TYPE, LENGTH,SIZE

		偏移地址
M1	?	0000H
	...	
	...	
M2	?	00C7H (199D)
	1	00C8H
	0	00C9H
	2	00CAH
	0	00CBH
	3	00CCH
	0	00CDH
M3	41H	00CEH
	42H	00CFH
	43H	00D0H
	44H	00D1H

```
MOV    BX, OFFSET M2
```

```
MOV    BX, SEG M2
```

```
MOV    AH, TYPE M1
```

```
MOV    AH, TYPE M3
```

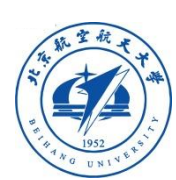
```
MOV    AH, LENGTH M1
```

```
MOV    AH, LENGTH M2
```

```
MOV    AH, SIZE M1
```

```
MOV    AH, SIZE M2
```

```
MOV    AH, SIZE M3
```



4.1.1 汇编语言程序格式--表达式和运算符

(4) 修改属性

- 包括段操作符、PTR, THIS, HIGH, LOW, SHORT、NEAR、FAR。
- 用来修改变量或标号的属性，如段属性、偏移地址属性、类型属性等等

例

```
N1      DB  15H, 36H
```

```
N2      DW  1122H, 3344H
```

```
MOV     AX, WORD PTR N1    ; AX=3615H
```

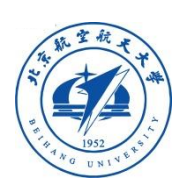
```
MOV     BL, BYTE PTR N2    ; BL = 22H
```



4.1.1 汇编语言程序格式--表达式和运算符

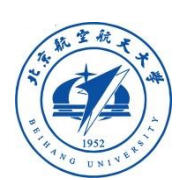
(5) 运算符优先级

优先级	运算符
1	(), [], < >, ., LENGTH, WIDTH, SIZE, MASK
2	PTR, OFFSET, SEG, TYPE, THIS, CS:, DS:, ES:, SS
3	HIGH, LOW
4	*, /, MOD, SHL, SHR
5	+, -
6	EQ, NE, LT, LE, GT, GE
7	NOT
8	AND
9	OR, XOR
10	SHORT



4.1.2 伪指令语句

- 段定义语句
- 段分配语句
- 过程定义语句
- 变量定义语句
- 表达式赋值语句
- 程序开始和结束语句



4.1.2 伪指令语句

(1) 段定义语句

格式：

段名 SEGMENT 定位类型 组合类型 '分类名'

逻辑段内容

段名 ENDS

- 段定义由伪操作**SEGMENT**开始、**ENDS**结束。
 - **SEGMENT** 和**ENDS** 必须成对出现
 - 语句前必须有段名，段名必须相同
- **SEGMENT**和**ENDS**语句之间可以包含有指令和其他伪操作
- 程序中可以定义多个段。
- 段定义语句允许嵌套，即逻辑段内允许设置其它逻辑段，但不允许逻辑段交叉设置



(1) 段定义语句

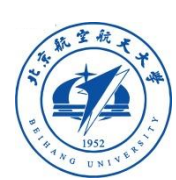
格式：

段名	SEGMENT	定位类型	组合类型	'分类名'
----	---------	------	------	-------

逻辑段内容

段名 ENDS

- 定位类型: PARA, BYTE, WORD, PAGE
- 组合类型: NONE, PUBLIC, COMMON, AT, STACK, MEMORY
- 分类名: 作用是在汇编的Link时将所有分类名相同的逻辑段组成一个段组。



4.1.2 伪指令语句

(2) 段分配

指定当前有效的逻辑段

格式: ASSUME CS:段名, DS: 段名

ASSUME SS: 段名, ES: 段名

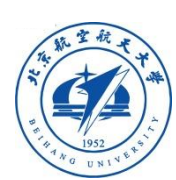
- 用户可以设置多个逻辑段，但只允许4个逻辑段同时有效
- 4种逻辑段不一定全部定义。只有代码段是必须的
- 可以使用NOTHING来取消前面逻辑段的定义，如
ASSUME ES:NOTHING

段定义和段分配举例：

```
DATA    SEGMENT
    D1    DW  1234H
    D2    DW  5678H
    P1    DD  ?
    P2    DD  ?
DATA                                ENDS
STK1     SEGMENT STACK 'STACK'
    DW    100 DUP(?)
STK1     ENDS
CODE     SEGMENT
    ASSUME CS:CODE, DS:DATA
    ASSUME SS:STK1
MAIN     PROC    FAR
START:   MOV     AX, STK1 ;初始化SS
        MOV     SS, AX
        MOV     SP, 100*2
        PUSH    DS      ;返回DOS用
```

```
                                SUB     AX,AX
                                PUSH    AX
                                MOV     AX, DATA ;初始化DS
                                MOV     DS, AX
L1:      MOV     AX, D1
                                MUL     D2      ;
                                MOV     BX, OFFSET P1 ;
                                MOV     [BX], AX
                                MOV     [BX+2], DX
                                RET
MAIN     ENDP
CODE     ENDS
END       START
```

*注： MUL D2 实现的运算： (DX,AX) = D1*D2



4.1.2 伪指令语句

(3) 过程定义语句

过程名	PROC 属性
	; 过程内容
	RET

过程名	ENDP
-----	------

- 过程名的三种属性：段属性，偏移地址属性，距离属性

例-近过程定义调用

CODE

SEGMENT

MAIN

PROC FAR

...

MOV CX,4

LL:

CALL ABC

LOOP LL

RET

MAIN

ENDP

ABC

PROC NEAR

LODSB

L1:

ADC AL, [BX]

DAA

STOSB

INC BX

RET

ABC

ENDP

CODE

ENDS

例 远过程定义及调用

```
MCODE      SEGMENT
MPA        PROC    FAR
... ..
```

```

RET
MPA      ENDP
```

```
MPB        PROC
... ..
```

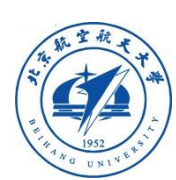
```
CALL    NEAR PTR MPA;同一段内调用(近调用)
... ..
```

```
MPB        ENPD
MCODE      ENDS
```

```
NCODE      SEGMENT                ;另一代码段
NPA        PROC
... ..
```

```
CALL    MPA                ;远过程调用
... ..
```

```
NPA        ENDP
NCODE      ENDS
```

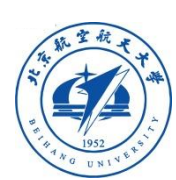


4.1.2 伪指令语句

(4) 变量定义语句

包括DB, DW, DD, DQ, DT, 分别定义1、2、4、8、10个字节的存储单元。

- 用DW定义字符串时, 只能包括2个字符。多于2个字符只能用DB
- 操作数用? 定义不确定值的变量
- 操作数DUP用来定义重复变量
- 用DW可以将变量或标号的偏移地址存入存储器



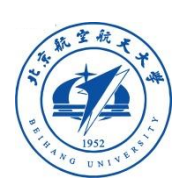
4.1.2 伪指令语句

(4) 变量定义语句

例：数据段段基址为2100H

```
DATA SEGMENT
    M1    DW    100 DUP(0)
    M2    DW    'AB',2, ?
    M3    DB    'ABCD'
    M4    DW    M2
DATA ENDS
```

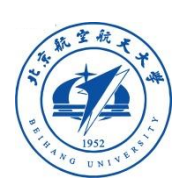
M1	0	0000H	
	...		
	...		
	0	00C7H	(199D)
M2	42H	00C8H	
	41H	00C9H	
	2	00CAH	
	0	00CBH	
	?	00CCH	
	?	00CDH	
M3	41H	00CEH	
	42H	00CFH	
	43H	00D0H	
	44H	00D1H	
M4	C8H	00D2H	
	00	00D3H	



4.1.2 伪指令语句

(5) 表达式赋值语句

- 包括EQU/PURGE
- 用来给变量，标号，常数，指令，表达式定义一个符号名
- PURGE用来取消EQU的定义



4.1.2 伪指令语句

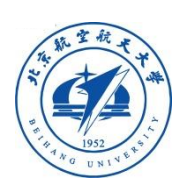
(5) 表达式赋值语句

例：

```
COUNT      EQU  100
DATE1      EQU  COUNT+2
MOV        SI, COUNT      ; MOV SI, 100
MOV        BX, DATE1      ; MOV BX, 102
```

```
A1          EQU  [BX+SI]
C1          EQU  ADD
```

```
MOV        AX, A1          ; MOV AX, [BX+SI]
C1         CX, AX          ; ADD CX, AX
PURGE      COUNT          ; COUNT 不再替代100
```



4.1.2 伪指令语句

(5) 程序定位、开始和结束语句

包括ORG, NAME, END

■ 格式: ORG 偏移地址

ORG后面的代码 (或数据) 从偏移地址处开始存放

例4.15

DATA

SEGMENT

ORG 1200H

A1

DB 12H, 34H

ORG 2000H

A2

DW 3040H, 2831H

DATA

ENDS

		偏移地址
A1	?	0000H
	...	
	...	
	12H	1200H
	34H	1201H
A2	...	
	...	
	40H	2000H
	30H	2001H
	31H	2002H
	28H	2003H



(5) 程序定位、开始和结束语句

包括ORG, NAME, END

- ## ■ 格式：ORG 偏移地址

ORG后面的代码（或数据）从偏移地址处开始存放

例：

CODE

SEGMENT

ORG 100H

ASSUME CS:CODE

START:

MOV AX, DATA

● ● ● ● ● ●

SUB1

PROC NEAR

● ● ● ● ● ●

SUB1

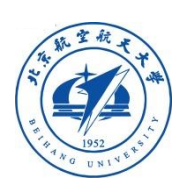
ENDP

CODE

ENDS

END START

[illegible]



4.1.2 伪指令语句

(5) 程序定位、开始和结束语句

包括ORG, NAME, END

- 格式: ORG 偏移地址

ORG后面的代码(或数据)从偏移地址处开始存放

- 格式: NAME 程序名

可以省略

- 格式: END 标号名

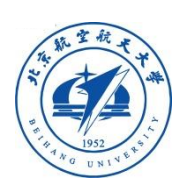
END放在源程序的**最后一行**。标号为主程序模块的第一条可执行语句的标号。



4.1.2 伪指令语句

例:

CODE	SEGMENT
	ORG 100H
	ASSUME CS:CODE
START:	MOV AX, DATA
	
SUB1	PROC NEAR
	
SUB1	ENDP
CODE	ENDS
	END START



4.1.3 汇编语言程序框架

- 一个完整的汇编程序包括：段定义、段分配、设置段地址、返回DOS语句以及程序结束，必要时还有过程调用和宏汇编。

1) 程序结构

程序可由多个段构成，至少有一个代码段

1. 段定义，段分配，设置段基址

- 每段由伪指令SEGMENT开始、由ENDS结束。
- 由ASSUME伪指令分配逻辑段
- 段寄存器赋值：DS、ES，SS。
- CS不能在程序中赋值。

2. 程序由数条语句构成，每条语句占一行。

- 指令语句，伪指令语句

3. 程序中设有返回DOS的功能。

4. 程序最后为END结束语句，后跟启动地址。

```
DATA    SEGMENT
        DA1 DB ' sample '

DATA    ENDS

STK1    SEGMENT STACK
        ST1  DB 100 DUP(?)

STK1    ENDS

CODE    SEGMENT
MAIN    PROC    FAR
        ASSUME CS:CODE
        ASSUME  DS:DATA
        ASSUME  SS:STK1

START:  MOV  AX, DATA
        MOV  DS, AX
        MOV  AH, 9
        MOV  DX, OFFSET DA1
        INT  21H
        MOV  AH, 4CH ;返回DOS
        INT  21H
        RET
MAIN    ENDP
CODE    ENDS
END     START
```

2) 程序框架

DATA SEGMENT

.....

DATA ENDS

STK1 SEGMENT STACK

ST1 DB 100 DUP(?)

STK1 ENDS

CODE SEGMENT

MAIN PROC FAR

ASSUME CS:CODE,DS:DATA,SS:STK1

START:

MOV AX, STK1

MOV SS, AX

MOV SP, SIZE ST1

MOV AX, DATA

MOV DS, AX

.....

MOV AH, 4CH ;返回DOS

INT 21H

RET

MAIN ENDP

CODE ENDS

END START

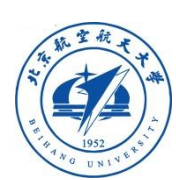
例 在屏幕上显示字符串 “This is a sample program.”

```
DATA    SEGMENT
        DA1 DB 'This is a sample program'
           DB 0DH, 0AH, '$'
DATA    ENDS

STACK   SEGMENT
        ST1   DB 100 DUP(?)
STACK   ENDS

CODE    SEGMENT
MAIN    PROC   FAR
        ASSUME CS:CODE, DS:DATA
        ASSUME SS:STACK
START:  MOV     AX, STACK
        MOV     SS, AX
        MOV     SP, SIZE ST1
```

```
        PUSH    DS           ;返回DOS
        MOV     AX, 0
        PUSH    AX
        MOV     AX, DATA
        MOV     DS, AX
        MOV     AH, 9
        MOV     DX, OFFSET DA1
        INT     21H
        RET
MAIN    ENDP
CODE    ENDS
END     START
```



4.1.3 汇编语言程序框架

3) 返回DOS操作系统

三种方法:

① 程序结束时, 执行4CH号DOS功能调用返回DOS

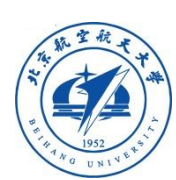
```
MOV    AH, 4CH  
INT     21H
```

② 主程序开始将DS和00入栈, 在程序结束时执行RET指令, 即可返回DOS

```
PUSH    DS  
SUB     AX, AX  
PUSH    AX  
... ..  
RET
```

③ 对于.COM文件, 可用INT 20H返回DOS

```
INT     20H
```



4.1.3 汇编语言程序框架

3) 返回DOS操作系统

三种方法：

①程序结束时，执行4CH号DOS功能调用返回DOS

```
MOV  AH, 4CH  
INT  21H
```



②主程序开始将DS和00入栈，在程序结束时执行RET指令，即可返回DOS

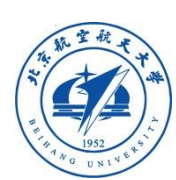
```
PUSH  DS  
SUB   AX, AX  
PUSH  AX  
... ..  
RET
```

- 程序中推荐使用第①种；
- 第②种形式也比较常见；
- 程序中①②两种返回DOS方式只写一种即可。



③对于.COM文件，可用INT 20H返回DOS

```
INT  20H
```



4.1.3 汇编语言程序框架

4) 堆栈设置

- 堆栈用于存放程序运行过程中需要保护的地址或数据。
 - 执行CALL指令时，用于保存返回地址
 - 响应中断时，用于保护断点和现场信息
- 可在程序中定义堆栈，在代码段中给SS和SP赋初值。
- 程序中未定义堆栈段，则由操作系统**自动**设置

4.1.3 汇编语言程序框架

5) 程序段前缀PSP (了解)

- ⑩ 操作系统在执行程序时 (*.exe或*.com文件)，自动为程序分配**程序段前缀PSP**(Program Segment Prefix)，长度256个字节，用于存储程序相关信息

PSP信息区的字段分布表

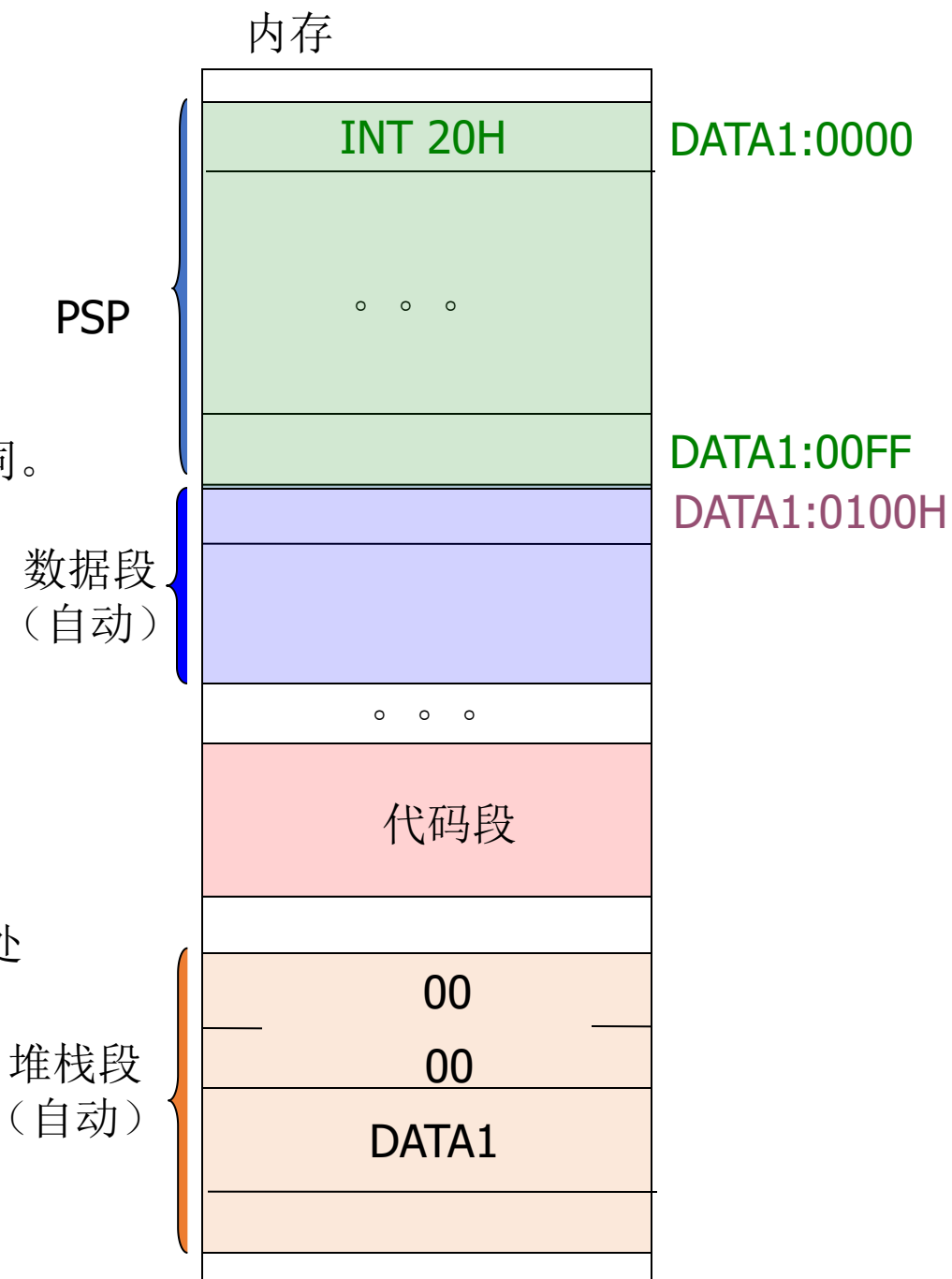
偏移量	内容含义	偏移量	内容含义
00~01H	程序结束指令中断20H	2E~31H	保留
02~03H	分配给该程序的最后段的段地址	32~33H	文件句柄表的长度
04~09H	保留	34~37H	指向文件句柄表的远指针
0A~0DH	中断22H的地址(处理终止程序)	38~4FH	保留
0E~11H	中断23H的地址(处理^Break)	50~51H	中断21H的功能调用
12~15H	中断24H的地址(处理严重错误)	52~5BH	保留
16~17H	保留	5C~6BH	参数区1
18~2BH	缺省的文件句柄表	6C~7FH	参数区2
2C~2DH	程序环境块的段地址	80~FFH	存储缺省DTA的缓冲区

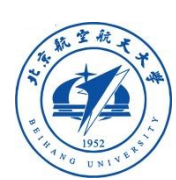
方法②的原理:

- 1.程序运行时，操作系统自动设置PSP，段基址为DATA1
- 2.操作系统设置默认数据区，并将段基址赋值给DS，段基址与PSP的段基址相同。
DS=DATA1

- 1.运行用户编写的程序，先执行
PUSH DS
SUB AX, AX
PUSH AX
- 2.程序结束，返回时执行RET指令
堆栈段中的 0000 -> IP, DATA1-> CS
则：程序转移到CS:IP(DATA1:0000)处
执行。

- 3.执行DATA1:0000处的指令INT 20H，
返回DOS





4.2 DOS系统功能调用

DOS操作系统的部分系统功能，能够以中断服务的方式供用户调用。用户可以使用**INT n**指令，调用相关的中断服务程序。

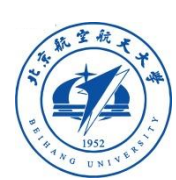
- $n=20H$ 时（即INT 20H）可返回DOS；
- $n=21H$ 时（即INT 21H），包含有大量子功能。每个子功能有自己的编号，称为**功能号**，调用前需要将功能号赋值给**AH**寄存器。
 - 键盘输入：1, 6, 7, 8, 0AH
 - 显示字符：2, 6, 9
 - 返回DOS操作系统：4CH



4.2 DOS系统功能调用--键盘输入

AH	功能	入口参数	出口参数
1	从键盘输入一个字符，并在屏幕上显示。检查Ctrl-Break键		AL=字符
6	直接键盘输入/输出字符，不检查Ctrl-Break键	DL=0FFH	AL=字符
0AH	输入字符串到内存缓冲区	DS:DX=缓冲区首地址	

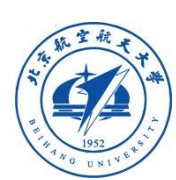
- 从控制台输入一个字符到AL中。1, 6, 7, 8号功能调用都可以实现，差别是1和6号在输入字符的同时在屏幕上显示，**7和8号功能不回显**



4.2 DOS系统功能调用--键盘输入

(1) 1号功能：从键盘输入字符并显示

- 入口参数 无
- 功能号 AH = 01H
- 类型号 21H
- 出口参数 AL = 键入字符的ASCII码
- 实现功能 等待从键盘输入一个字符,并在屏幕上回显, 光标移动, 检测Ctrl-Break键



4.2 DOS系统功能调用--键盘输入

(1) 1号功能：从键盘输入字符并显示

例4.19 利用键盘输入的字符产生分支

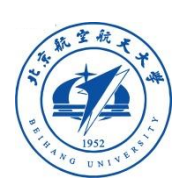
```
KEY:  MOV    AH, 1           ;从键盘输入
      INT     21H
      CMP     AL, 'Y'        ;是' Y' ?
      JE      YES
      CMP     AL, 'N'        ;是' N' ?
      JE      NO
      JNE     GET_KEY        ;不是N，返回继续等待
      JMP     KEY

YES:  ...

      ...

NO:   ...

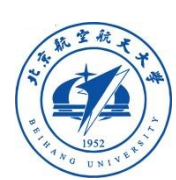
      ...
```



4.2 DOS系统功能调用--键盘输入

(2) 6号功能：直接控制台输入/输出

- 入口参数 DL=0FFH
 - 功能号 AH = 06H
 - 类型号 21H
 - 出口参数 若ZF=0, 则AL = 键入字符的ASCII码;
ZF=1表示无键按下
 - 实现功能 从键盘输入一个字符,并在屏幕上回显,
光标移动, 不检测Ctrl-Break键、不等待
- DL=0FFH表示从键盘输入, 否则是向显示器输出



4.2 DOS系统功能调用--键盘输入

(2) 6号功能：直接控制台输入/输出

- 入口参数 DL=0FFH
- 功能号 AH = 06H
- 类型号 21H
- 出口参数 若ZF=0, 则AL为按下键的ASCII码
ZF=1表示无键按下
- 实现功能 从键盘输入一个字符并显示在屏幕上
光标移动, 不检测Ctrl-Break

例：

```
CHR DB ?
```

... ..

```
AGN: MOV DL, 0FFH
```

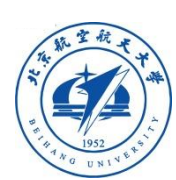
```
MOV AH, 06H
```

```
INT 21H
```

```
JZ AGN
```

```
MOV CHR, AL
```

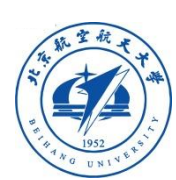
➤ DL=0FFH表示从键盘输入，否则是同显示器输出



4.2 DOS系统功能调用--键盘输入

(3) 0A号功能：从键盘接收字符串存入缓存区

- **入口参数** $DS : DX =$ 缓冲区的首地址
 $(DS : DX) =$ 最多输入的字符数
- **功能号** $AH = 0AH$
- **类型号** $21H$
- **出口参数** $(DS : DX + 1) =$ 实际键入的字符(不含回车符)
 从 $(DS : DX + 2)$ 开始顺序存放键入的字符串
 回车符 $0DH$ 为串尾最后一字符。
- **实现功能** 等待从键盘输入字符串，存入设定的缓冲区内，
 同时回显字符串，光标随着移动，回车符使光标回到行首。
- **注意事项：** 应按要求先定义缓冲区，再调用。



4.2 DOS系统功能调用--键盘输入

(3) 0A号功能：从键盘接收字符串存入缓存区

- 入口参数 $DS : DX =$ 缓冲区的首地址
 $(DS : DX) =$ 最多输入的字符数

- 功能号 $AH = 0AH$

- 缓存区要求：
 - 第一个字节：指明最大能容纳的字符数
 - 第二个字节：实际输入的字符数
 - 剩余字节：存放从键盘输入的字符，直到Enter结束
- $DS:DX$ 指向缓存区的第一个字节

同时回显字符串，光标随有移动，回车符使光标回到行首。

- 注意事项：应按要求先定义缓冲区，再调用。



例4.21

```
DATA SEGMENT
BUFF DB 100           ;定义缓冲区, 长度100
DB ?
DB 100 DUP(?)
DATA ENDS

CODE SEGMENT
MAIN PROC FAR
ASSUME CS:CODE, DS:DATA
START: ... ..
    MOV AX, DATA
    MOV DS, AX
    MOV DX, OFFSET BUFF
    MOV AH, 0AH         ;0AH#
    INT 21H
    ... ..
```

- 缓存区要求:
 - 第一个字节: 指明最大能容纳的字符数
 - 第二个字节: 实际输入的字符数
 - 剩余字节: 存放从键盘输入的字符, 直到Enter结束
- DS:DX指向缓存区的第一个字节

BUFF	64H	0000H
	?	0001H
	?	0002H
	?	0003H
	?	0004H
	?	0005H
	?	0006H
	?	0007H
	...	
	?	0065H



例4.21

- 缓存区要求:
 - 第一个字节: 指明最大能容纳的字符数
 - 第二个字节: 实际输入的字符数

假设从键盘输入 'ABCD' , 回车。
则内存的存放结果:

DATA SEGMENT

BUFF DB 100 ;定义缓冲区, 长度100

DB ?

DB 100 DUP(?)

DATA ENDS

CODE SEGMENT

MAIN PROC FAR

ASSUME CS:CODE, DS:DATA

START:

MOV AX, DATA

MOV DS, AX

MOV DX, OFFSET BUFF

MOV AH, 0AH ;0AH#

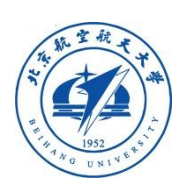
INT 21H

... ..

实际输入字符数

回车符

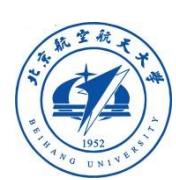
BUFF	64H	0000H
	04H	0001H
	41H	0002H
	42H	0003H
	43H	0004H
	44H	0005H
	0DH	0006H
	?	0007H
	...	
	?	0065H



4.2 DOS系统功能调用--键盘输入

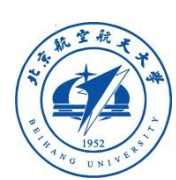
0AH功能注意事项

- 回车符0DH作为一个输入的字符存放在字符串尾，但计数输入个数时，不包括回车键。
 - 所以实际最多能输入的字符数 = 定义的最多数-1 (回车符占一个)
- 整个缓冲区的大小应为: 定义的最多数+2
- 执行完0AH功能后，DS和DX的值不变，DS:DX仍指向缓冲区的首地址。
- 当按下键的个数超过(DS : DX)中的值，发出警告声‘嘟嘟’，不再接收输入的数据，直到输入回车键



4.2 DOS系统功能调用--显示输出

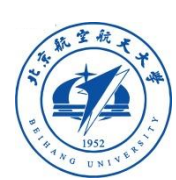
AH	功能	入口参数	出口参数
2	显示一个字符，检验 Ctrl-Break键	DL=字符	光标跟随字符 移动
6	显示一个字符，不检 验Ctrl-Break键	DL=字符	光标跟随字符 移动
9	显示字符串	DS:DX=串 地址	串以'\$'结束， 光标跟随字符 移动



4.2 DOS系统功能调用--显示输出

1) 2号功能：将字符送到屏幕显示

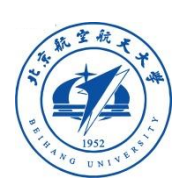
- 入口参数 DL = 要显示字符的ASCII 码
- 功能号 AH=02H
- 类型号 INT 21H
- 出口参数 无
- 实现功能 显示指定字符，光标随动。



4.2 DOS系统功能调用--显示输出

1) 2号功能：将字符送到屏幕显示

例：编写子程序KEYIN，
等待从键盘输入一个数
字字符、显示并保存在
AL寄存器。如果输入非
数字字符则不显示。



4.2 DOS系统功能调用--显示输出

2) 6号功能：直接控制台输入输出

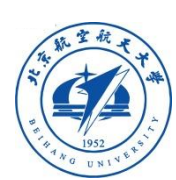
- 入口参数 DL = 要显示字符的ASCII 码
(不能是FFH)
- 功能号 AH=06H
- 类型号 INT 21H
- 出口参数 无

例：6号功能调用

```
MOV AH, 6
MOV DL, 'S'
INT 21H
```

例：2号功能调用

```
MOV AH, 2
MOV DL, 'S'
INT 21H
```



4.2 DOS系统功能调用--显示输出

3) 9号功能：显示字符串

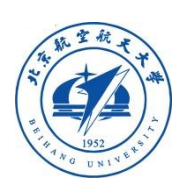
- 入口参数 在内存中定义要显示的字符串
字符串尾必须为 ' \$ '
DS : DX = 字符串的首地址
- 功能号 AH = 09 H
- 类型号 21H
- 出口参数 无
- 实现功能 显示字符串，遇 ' \$ ' 停止显示，光标随动。

9号功能调用示例:

- 在内存中定义要显示的字符串，字符串尾必须为 '\$'；
- DS : DX = 字符串的首地址

```
DATA    SEGMENT
        CR      EQU  0DH
        LF      EQU  0AH
        MES     DB  'HOW DO YOU DO?',CR,LF,'$'
DATA    ENDS
CODE    SEGMENT
MAIN    PROC    FAR
        ASSUME  CS:CODE, DS:DATA
START:  ... ..
        MOV     AX, DATA
        MOV     DS, AX

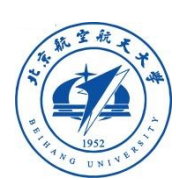
        MOV     DX, OFFSET MES
        MOV     AH, 9
        INT     21H
        ....
        RET
MAIN    ENDP
CODE    ENDS
        END     START
```



4.3 程序设计方法

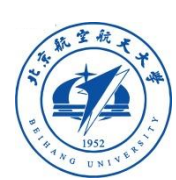
程序设计步骤一般为：

- 分析问题，得到数学模型或者算法
- 绘制程序流程图
- 分配存储空间
- 按流程图编写程序
- 程序编译、调试
- 运行程序，分析结果



汇编语言程序设计通常用到4种程序结构：

- 顺序结构
- 分支结构
- 循环结构
- 子程序结构



4.3.1 顺序结构程序设计

- 程序按指令排列的先后顺序逐条执行

例4.33

```
PROG1      SEGMENT
            ASSUME      CODE:PROG1

START:      MOV         DL, 1           ; ‘☺’ → DL
            MOV         AH, 2
            INT         21H           ;显示 ‘☺’
            MOV         AX, 4C00H
            INT         21H

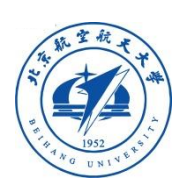
PROG1      ENDS
            END  START
```

例: 编程完成两个32位无符号数相加: 9977 6543H+FEDC 1122H

```
DATA        SEGMENT
A1          DW    9977H, 6543H
A2          DW    FEDCH, 1122H
SUM         DW    3 DUP(0)
DATA        ENDS
```

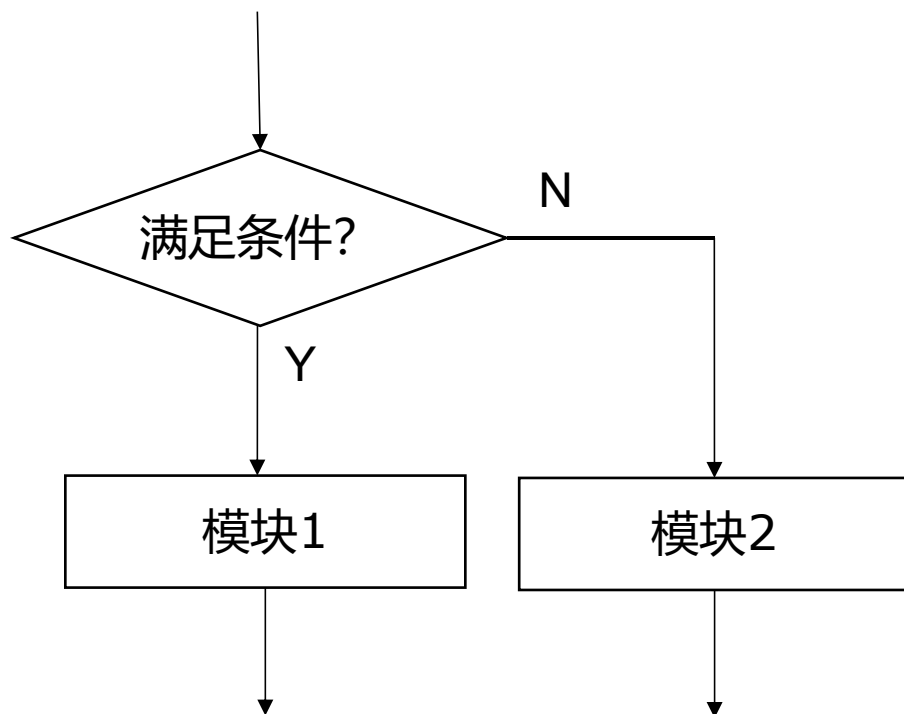
编程思路:

- (1) 低16位相加 (**ADD**)，保存至SUM+4
- (2) 高16位相加 (**ADC**)，保存至SUM+2
- (3) 保存高16位相加产生的进位位 (**CF**) 至SUM
- (4) 其它: 写出完整的程序结构 (代码段定义、段分配、段寄存器初始化、返回DOS)



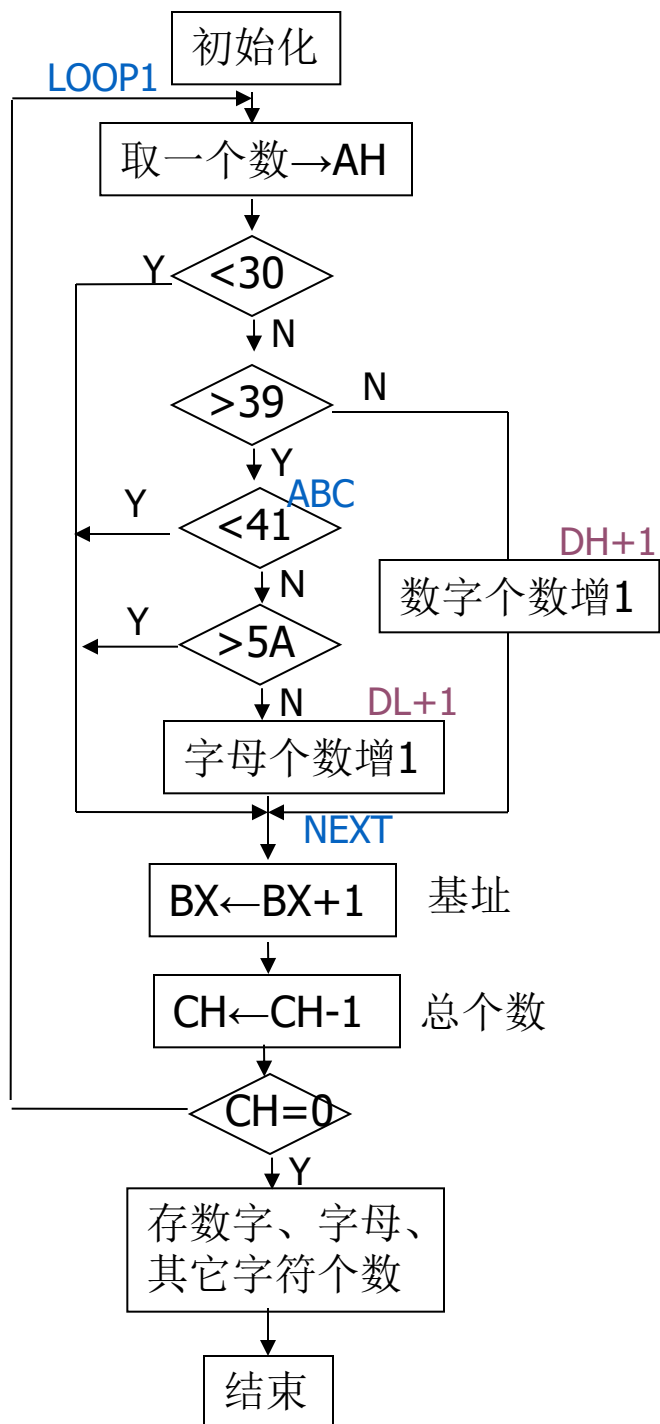
4.3.2 分支程序设计

- 程序根据不同的条件选择不同的处理方法



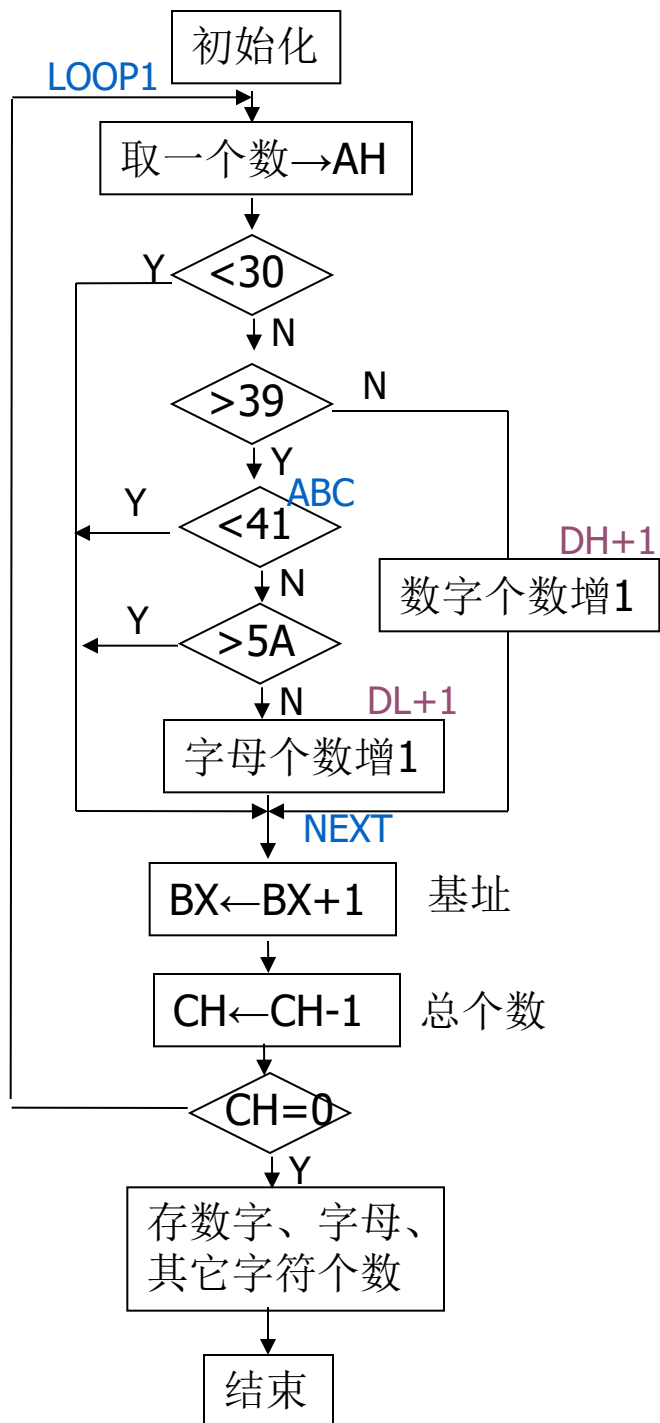
例4.37 数据段中从BUF开始存放一串字符，长度为COUNT。编程统计 '0' ~ '9' ，字母A~Z，以及其它字符的个数，并保存到NUM开始的3个内存单元中。

DATA	SEGMENT
BUF	DB 'PRINT','abc',35H,52H,30H,08H
COUNT	EQU \$-BUF
NUM	DB 3 DUP(?)
DATA	ENDS

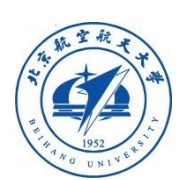


DATA	SEGMENT	
BUF	DB	'PRINT','abc',35H,52H,30H,08H
COUNT	EQU	\$-BUF
NUM	DB	3 DUP(?)
DATA	ENDS	

CODE	SEGMENT	
	ASSUME	CS:CODE, DS:DAT
START:	MOV	AX, DATA
	MOV	DS, AX
		• • •
		• • •
		• • •
	MOV	AH, 4CH
	INT	21H
CODE	ENDS	
	END	START

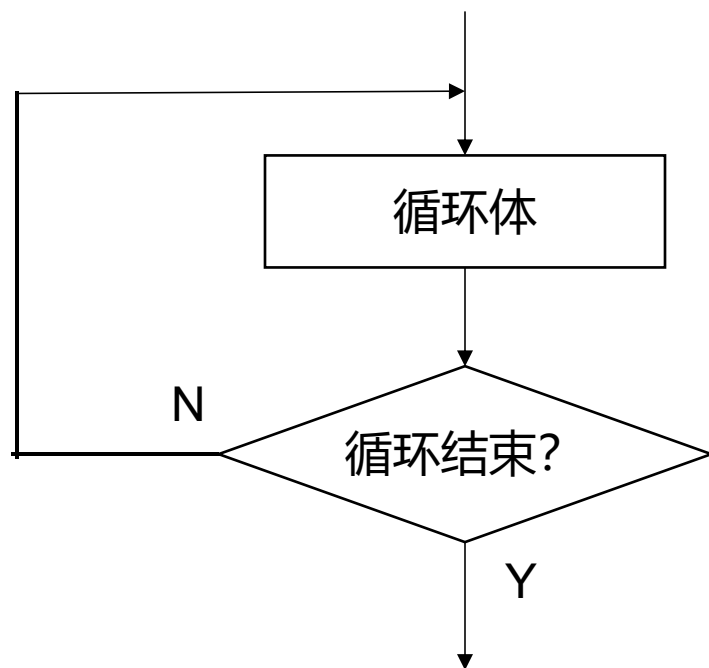


DATA	SEGMENT	
BUF	DB	'PRINT','abc',35H,52H,30H,08H
COUNT	EQU	\$-BUF
NUM	DB	3 DUP(?)



4.3.3 循环结构

- 某段程序需要反复执行多次，直到满足某些条件时为止，称为循环结构程序。



例4.38 在一串给定个数的数据中寻找最大值，存放到MAX中。

DATA SEGMENT

BUF DW 1234H, 3200H, 4832H, 5600H

COUNT EQU (\$-BUF)/2

MAX DW ?

DATA ENDS

CODE SEGMENT

MAIN PROC FAR

ASSUME CS:CODE, DS:DATA

START: MOV AX, DATA

MOV DS, AX

... ; 循环结构/循环体

MOV AH, 4CH

INT 21H

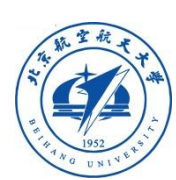
RET

MAIN ENDP

CODE ENDS

例4.38 在一串给定个数的数据中寻找最大值，存放MAX中。

```
DATA      SEGMENT
BUF        DW      1234H, 3200H, 4832H, 5600H
COUNT     EQU     ($-BUF)/2
MAX         DW      ?
DATA      ENDS
```



4.3.5 过程调用

- 某些完成特定功能而且经常用到的程序段，编写为独立的模块，称之为过程或者子程序。
- 需要执行子程序时就用CALL指令调用，执行完毕后返回到原来调用它的主程序中。

例：从键盘输入一个三位数的数字，分别转换为非压缩BCD码和二进制码存储到DT1和DT2中。

DT1 DB 3 DUP(?)

DT2 DW ?

1. 调用KEYIN输入数字，转换为BCD码存储到DT1；
2. 转换为二进制数：
 - 1) 输入第一个数，将其BCD码直接存入DT2
 - 2) 输入第二个数，其BCD码 + DT2*10
 - 3) 输入第三个数，其BCD码 + DT2*10

附:

KEYIN PROC NEAR

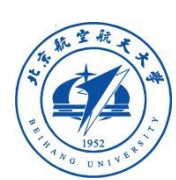
AGAIN: MOV AH, 8 ; 输入字符, 但不显示
INT 21H

CMP AL, 30H ; '0'
JB AGAIN

CMP AL, 39H ; '9'
JA AGAIN

MOV DL, AL ; 显示刚才输入的字符
MOV AH, 2
INT 21H
RET

KEYIN ENDP



第四章 总结

- 汇编程序框架
- 伪指令和表达式
- DOS和BIOS功能调用
- 基本程序结构